RLFORGE

RL Environment Generation & Training Platform

Implementasyon Kılavuzu v5.0

Mart 2026 | Tüm bölümler dahil

KULLANIM AMACI

Bu doküman RLForge platformunu kodlayacak LLM/geliştirici için implementasyon kılavuzudur. TÜM özellikler kapsam dahilindedir (Kubernetes ve gRPC hariç). Architect Agent'ın tüm prompt'ları ve skill'leri bu dokümanda tam olarak tanımlıdır.

İÇİNDEKİLER

B1: Sistem Özeti | B2: Fikir Giriş | B3: Architect Agent | B4: Data | B5: Görselleştirme | B6: Hosting | B7: Training API | B8: Fizibilite | B9: Maliyet | B10: Fiyatlandırma | B11: Environment Builder | B12: Agent Builder | B13: AI Research Lab | B14: Yol Haritası | B15: Riskler | EK A: Landing Page

Bölüm 1: Sistem Özeti ve Kapsam

Platform Akışı
Fikir Gir → Architect Agent Env Üretir → Dashboard'da Gör → AI ile Iterate Et → Beğenince GitHub'a Çek → (Opsiyonel) Agent Eğit

1.1 Kapsam (Tüm Özellikler)

HER ŞEY KAPSAM DAHİLİNDE
Aşağıdaki tüm özellikler kodlanacaktır. Kubernetes ve gRPC kapsam dışıdır.

- Fikir giriş sistemi (NL → env spec)
- Architect Agent (kod üretimi + test + iteratif düzeltme + dinamik skill)
- Otomatik yayınlama (Docker + API endpoint)
- Training API Mod A: Remote Step
- 5-10 hazır şablon environment
- Python SDK
- Tek sunucu hosting (Hetzner)
- State Viewer dashboard
- Environment Builder (Lovable modeli: AI ile iterate, dashboard, GitHub export)
- Agent Training ("Bu env'de agent eğit" butonu, SB3)
- Agent Builder (Lovable for Agents: AI ile agent tasarla, iterate, sürekli geliştir, curriculum, transfer)
- Paper Upload (PDF → env spec → iterate)
- AI Research Lab (Literature, Replication, Experiment, Analysis, Benchmark agent'ları)
- Canvas 2D render (basit ortamlar için)
- WebSocket modu (düşük latency)
- Leaderboard ve benchmark sistemi
- Mod B: Agent Upload (kullanıcı agent kodunu yükler, içeride çalıştırır)

KAPSAM DIŞI

Kubernetes ve gRPC bu süreçte KODLANMAYACAKTIR.

1.2 Tek Architect Agent

Tek Agent, İki Mod

Aynı Architect Agent hem platformun hazır env'lerini (batch) hem kullanıcının custom env'lerini (on-demand) üretir. Fark sadece tetikleyicide: platform toplu üretim için cron/batch job çalıştırır, kullanıcı ise API üzerinden tek tek istek gönderir. Agent kodu, prompt'ları, test pipeline'ı aynıdır.

Bölüm 2: Fikir Giriş Sistemi

Kullanıcı tek metin alanına istediği RL ortamını doğal dille anlatır.

2.1 Girdi Yöntemleri

- **Serbest metin:** “Borsa trading ortamı, 5 hisse, komisyon %0.1”
- **Şablon + özelleştirme:** Domain seç (finans/oyun/robotik), parametreleri ayarla
- **Fork:** Mevcut ortamı al, üzerine değişiklik yap

2.2 Doğal Dil → Environment Spec JSON

```
{
  "name": "stock-trading-5",
  "domain": "finance",
  "observation_space": { "type": "Box", "shape": [30] },
  "action_space": { "type": "Box", "shape": [5], "low": -1, "high": 1 },
  "reward_function": { "type": "portfolio_return", "commission": 0.001 },
  "episode": { "max_steps": 252 }
}
```

2.3 Akıllı Varsayılanlar

- **Finans:** commission=0.001, episode=252, obs=OHLCV
- **Robotik:** physics=2D, dt=0.02, max_steps=1000
- **Oyun:** grid=10x10, vision=full, reward=sparse

Bölüm 3: Architect Agent — Tam Prompt & Skill Tanımı

> **Bu Bölüm Neden Kritik?**
> Architect Agent, platformun kalbidir. Bu bölüm agent'ın tüm system prompt'larını, skill'lerini, araçlarını ve karar mantığını tam olarak tanımlar. Bu bölüm, agent'ı kodlayacak geliştirici için kesin referanstır.

3.1 Agent'ın Rolleri

Rol	Tetikleyici	Çıktı
Env Generator	Kullanıcı veya batch job env ister	Gymnasium Python kodu
Env Tester	Kod üretildiğinde	Test sonuçları (pass/fail + detay)
Env Improver	Test fail veya metrik anomali	Patch (düzeltilmiş kod)
Env Deployer	Test geçtiğinde	Docker image + API endpoint
Env Documenter	Her versiyon değişikliğinde	Changelog, API docs

3.2 System Prompt (Ana Prompt)

Architect Agent'ın her çağrıda aldığı sabit system prompt:

```
SYSTEM PROMPT - RLFORGE ARCHITECT AGENT
========================================

Sen RLForge platformunun Architect Agent'isin.
Gorevlerin: RL environment uretmek, test etmek, iyilestirmek,
deploy etmek ve dokumante etmek.

## TEMEL KURALLAR
1. SADECE Gymnasium v0.29+ uyumlu Python kodu uret.
2. Her environment gymnasium.Env sinifindan turetilmeli.
3. Zorunlu metodlar: __init__, step, reset, close
4. step() donusu: (observation, reward, terminated, truncated, info)
5. reset(seed=None) donusu: (observation, info)
6. observation_space ve action_space __init__'te tanimlanmali
7. Tum numerik degerler numpy float32 olmali
8. np.random.Generator kullan (eski np.random KULLANMA)
9. render() metodu opsiyonel ama tavsiye edilir

## KOD KALITE KURALLARI
1. Type hints kullan
2. Docstring yaz (sinif ve her public metod icin)
3. Magic number kullanma, sabitleri sinif seviyesinde tanimla
```

```
4. Edge case'leri handle et (NaN, Inf, out-of-bounds)
5. Seed ile deterministik olmayi garanti et

## REWARD FONKSIYONU KURALLARI
1. Reward degerlerini [-10, 10] araliginda tut (clipping)
2. Dense reward tercih et (sparse sadece zorunluysa)
3. Negative reward, positive ile orantili olmali
4. Reward hacking'e karsi koruma ekle
5. info dict'ine reward bilesenleri ayri ayri yaz (debug icin)

## OBSERVATION SPACE KURALLARI
1. Tum degerleri [-1, 1] veya [0, 1] araligina normalize et
2. Sadece karar icin gerekli bilgiyi dahil et
3. Gymnasium standart tiplerini kullan: Box, Discrete, MultiDiscrete

## ACTION SPACE KURALLARI
1. Discrete: sonlu secenekler (al/sat/bekle)
2. Box: surekli degerler (portfoy agirliklari)
3. Gecersiz aksiyonlari icsel olarak handle et

## CIKTI FORMATI
Sadece Python kodu uret. Aciklama, markdown, yorum blogu YAZMA.
Kod calisir durumda olmali - import'lar dahil.
```

3.3 Skill Sistemi (İki Katmanlı: Hazır + Dinamik)

Tasarım Prensibi

Architect Agent sınırlı sayıda domain'e kilitli DEĞİLDİR. İki katmanlı skill sistemi ile sonsuz sayıda domain destekler. Katman 1: sık kullanılan domain'ler için hazır, optimize edilmiş skill prompt'ları. Katman 2: bilinmeyen domain'ler için LLM'in anlık ürettiği dinamik skill prompt'ları.

3.3.1 Skill Seçim Akışı

```
SKILL SECIM AKISI:

1. Kullanici istegi gelir: 'havalimani bagaj bandi optimizasyonu'

2. Domain classifier calisir:
   - Bilinen domain mi? (finans, grid, kontrol, optimizasyon, oyun)
   - EVET -> Hazir skill prompt'u kullan (Katman 1)
   - HAYIR -> Dinamik skill generation (Katman 2)

3. Katman 2 - Dinamik Skill Generation:
   LLM'e sor: 'Havalimani bagaj bandi optimizasyonu icin
```

```
   bir RL environment yazarken dikkat edilmesi gereken
   domain-spesifik kurallar nelerdir?'

   LLM cevabi (ornek):
   - Observation: bant hizi, kuyruk uzunlugu, bagaj boyutlari
   - Action: bant hizi ayarla, yonlendirme kapisi ac/kapa
   - Reward: throughput - bekleme_suresi - hasar_orani
   - Constraints: max bant hizi, kapasite limiti
   - Termination: tum bagajlar islendi veya timeout

4. Bu dinamik skill, system prompt'a eklenir
5. Kod uretimi bu enriched context ile yapilir
```

3.3.2 Katman 1: Hazır Skill Prompt'ları

Sık kullanılan domain'ler için önceden test edilmiş, optimize edilmiş prompt ekleri. Bu skill'ler daha yüksek kalite garantisi verir çünkü domain-spesifik kurallar ve edge case'ler önceden tanımlanmıştır.

Hazır Skill	Kapsam	Temel Kurallar (Özet)
Finans	Trading, portfolio, market making	OHLCV obs, komisyon, risk penalty, episode=252, bankrupt=terminate
Grid/Maze	Navigasyon, pathfinding, key-door	Discrete(4) action, duvar/engel, sparse/dense reward, seed-based grid
Klasik Kontrol	CartPole, Pendulum, MountainCar	Euler physics, dt parametrik, alive bonus, angle/pos threshold
Optimizasyon	TSP, bin packing, scheduling	Constraint violation penalty, minimize cost, episode=iş bitti
Oyun AI	Snake, Pong, multi-agent	Skor=reward, ölüm=terminate, frame stacking opsiyonel

Bu skill'ler başlangıç seti. Platform büyüdükçe yeni hazır skill'ler eklenir: sağlık, enerji, telekom, NLP, robotik navigasyon, supply chain vb. Dinamik skill'lerden en sık kullanılanlar zaman içinde hazır skill'e promote edilir.

3.3.3 Katman 2: Dinamik Skill Generation

Kullanıcı bilinmeyen bir domain istediğinde, Architect Agent otomatik olarak o domain için geçici bir skill prompt üretir:

```
DINAMIK SKILL GENERATION PROMPT:
```

```
Sen bir RL environment tasarim uzmansin.
Kullanici su domain icin bir environment istiyor:

DOMAIN: {user_description}

Bu domain icin bir RL environment yazarken dikkat edilmesi
gereken domain-spesifik kurallari listele:

1. OBSERVATION SPACE: Bu domain'de agent ne gormeli?
   Hangi degerler/metrikler karar icin kritik?

2. ACTION SPACE: Agent ne yapabilmeli?
   Discrete mi continuous mu? Kac boyutlu?

3. REWARD FUNCTION: Basari nasil olculur?
   Hangi metrikler odullendirilmeli, hangisi cezalandirilmali?

4. TRANSITION DYNAMICS: Fizik/kurallar nasil calisir?
   Deterministik mi stokastik mi?

5. TERMINATION CONDITIONS: Episode ne zaman biter?
   Basari kosulu? Basarisizlik kosulu? Zaman asimi?

6. EDGE CASES: Bu domain'e ozel dikkat edilmesi
   gereken sinir durumlar neler?

7. TIPIK PARAMETRELER: Varsayilan degerler ne olmali?

SADECE kurallari listele. Kod yazma.
```

LLM'in ürettiği domain kuralları, system prompt'a eklenir ve kod üretimi bu zenginleştirilmiş context ile yapılır. Bu dinamik skill prompt'lar cached edilir — aynı domain tekrar istenirse yeniden üretilmez.

3.3.4 Skill Gelişim Döngüsü

```
SKILL YASAM DONGUSU:

Bilinmeyen domain istegi
    |
    v
Dinamik skill uretilir (Katman 2)
    |
    v
```

```
Env uretilir, test edilir, deploy edilir
    |
    v
Skill cache'e kaydedilir (ayni domain icin reuse)
    |
    v
[Eger bu domain 10+ kez istenirse]
    |
    v
Admin review -> Hazir skill'e promote et (Katman 1)
    |
    v
Skill prompt optimize edilir, edge case'ler eklenir
```

Kriter	Katman 1 (Hazır)	Katman 2 (Dinamik)
Kalite	Yüksek (önceden optimize)	Orta (LLM'e bağlı)
Hız	Anlık (cache)	2-5 sn ek (skill üretim süresi)
Domain kapsamı	5-20 bilinen domain	Sonsuz (her şey)
Edge case	Tanımlı, test edilmiş	LLM'in bilgisine bağlı
Maliyet	Ek LLM çağrısı yok	+1 LLM çağrısı (~$0.02)

3.4 Araçlar (Tools)

Araç Adı	Fonksiyon	Input	Output
classify_domain	Girdinin bilinen domain'e uyup uymadığını belirle	User description	domain_id veya 'unknown'
generate_skill	Bilinmeyen domain için dinamik skill prompt üret	User description	Domain kuralları (skill prompt)
generate_code	LLM'e spec + skill prompt gönder, Gymnasium kodu al	env_spec JSON + skill prompt (hazır veya dinamik)	Python kodu (string)
parse_ast	Kodu AST ile parse et, syntax kontrol	Python kodu	AST tree veya hata listesi
check_api	Gymnasium API uyumu kontrol (metodlar, return tipleri)	Python kodu	Pass/fail + eksik metodlar
run_smoke	Docker sandbox'ta 1000 random step çalıştır	Python kodu + seed	Step sonuçları, crash log
check_reward	Reward dağılımı analiz (NaN, Inf, sabit değer)	1000 step reward dizisi	Pass/fail + istatistik

Araç Adı	Fonksiyon	Input	Output
check_obs	Observation'ların space içinde olduğunu doğrula	1000 step obs dizisi	Pass/fail + out-of-range
check_determinism	Aynı seed = aynı sonuç mü	Python kodu + seed	Pass/fail
check_perf	Step/saniye ölç	Python kodu	step/sec sayısı
check_memory	10K step RAM kullanımı	Python kodu	MB cinsinden peak RAM
build_docker	Kodu Docker image'a paketle	Python kodu + requirements	Docker image ID
deploy	Container'ı başlat, API endpoint aç	Docker image ID	Endpoint URL
rollback	Son deploy'u geri al	env_id + version	Geri alınan versiyon
patch_code	Mevcut koda diff-based değişiklik uygula	Orijinal kod + patch talimatı	Güncellenmiş kod

3.5 Karar Matrisi

Durum	Agent Aksiyonu	Kullanılan Araçlar
Yeni env isteği geldi	Domain classify et, hazır veya dinamik skill seç/üret, kod üret	classify_domain + (generate_skill?) + generate_code
Kod üretildi	Tüm test pipeline'ı çalıştır	check_api, run_smoke, check_reward, check_obs, check_determinism, check_perf, check_memory
Test fail	Hata mesajıyla yeniden üret (maks 3 deneme)	generate_code (hata context ile)
Tüm testler geçti	Docker build + deploy	build_docker + deploy
Crash rate > %5	Acil bug fix	patch_code + run_smoke
Reward hacking tespit	Reward fonksiyonunu düzelt	patch_code + check_reward
Success rate > %80	Difficulty artır	patch_code (parametre ayarı)
Major change gerekli	İnsan onayı iste	Onay bekle

3.6 Hata Düzeltme Prompt'u

Test fail ettiğinde agent'a gönderilen ikinci prompt:

```
HATA DUZELTME PROMPT:


Onceki kodun asagidaki testleri GECEMEDI:
```

```
FAIL: {test_name}
HATA: {error_message}
DETAY: {error_details}

Orijinal spec: {env_spec_json}
Onceki kod: {previous_code}

HATALI BOLUMU DUZELT ve TUM KODU tekrar uret.
Sadece Python kodu uret. Aciklama yazma.
```

3.7 Batch vs On-Demand Üretim

Kriter	Platform Batch (Hazır Env'ler)	Kullanıcı On-Demand
Tetikleyici	Cron job / admin komutu	Kullanıcı API isteği
Agent	Aynı Architect Agent	Aynı Architect Agent
Prompt	Aynı system + skill prompt	Aynı system + skill prompt
Test pipeline	Aynı 8 test	Aynı 8 test
Fark	Spec platform tarafından yazılır	Spec kullanıcı girdisinden üretilir
Kalite kontrol	Ekstra manual review	Otomatik (test geçerse publish)

Bölüm 4: Data Meselesi

MVP'de sadece kural-tabanlı env'ler

Çoğu env'in gerçek veriye ihtiyacı yoktur. Grid world, cart-pole, maze, optimizasyon — hepsi matematiksel kurallarla çalışır. Agent kendi datasını milyonlarca step atarak üretir. Data-driven env'ler (finans, sağlık) Phase 2'de halka açık veri ile gelir.

Transfer Learning Sınırlaması

Farklı env'lerde eğitilen agent'lar birbirine transfer edilemez. Farklı obs/action space = farklı dünya.

Bölüm 5: Görselleştirme

> **MVP: Sadece State Viewer**
> Canvas/WebGL KODLANMAYACAK. JSON state tablosu + reward grafik + action log yeterli.

Bölüm 6: Hosting

Her env = bir Docker container. MVP: Hetzner CPX31 ($18/ay) tek sunucu.

```
Cloudflare -> Nginx -> FastAPI Gateway -> Docker Engine
                                       -> PostgreSQL + Redis
```

- **Cold start:** 2-5 sn. Idle timeout: 15 dk. LRU eviction.
- **Güvenlik:** Container izolasyonu, no egress, resource limits, kod tarama.

Bölüm 7: Training API (Mod A: Remote Step)

> **MVP Modu**
> Agent kullanıcı makinesinde çalışır, API üzerinden step atar. Mod B = Phase 2.

Endpoint	Metod	Açıklama
/v1/envs	GET	Environment listesi
/v1/envs/{id}	GET	Env detayı
/v1/sessions/create	POST	Oturum başlat
/v1/sessions/{sid}/reset	POST	Reset
/v1/sessions/{sid}/step	POST	Step
/v1/sessions/{sid}/close	POST	Kapat

```
import rlforge

env = rlforge.make('gridworld-10x10', api_key='KEY')
obs, info = env.reset(seed=42)
for step in range(10000):
    action = my_agent.act(obs)
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        obs, info = env.reset()
```

Bölüm 8: Fizibilite

Akademik Kanıt

NVIDIA Eureka: 29 ortamda %83 insanı geçti. Text2Reward: 13/17 görevde insan seviyesi. CARD: otonom iteratif iyileştirme. Teknoloji çalışıyor, henüz kimse ürünleştirmedi.

Gerçek Dünya Validation

X/Twitter'da bir RL araştırmacısı: "I need an IDE for RL research. I'm vibe coding UIs for each paper and environment, but I want reusable primitives... without me having to find the repo, download the dataset and prompt." Bu kullanıcının istediğinin %60-70'ini RLForge karşılıyor.

Bölüm 9: Maliyet

Senaryo	Env/ay	Sabit	LLM	TOPLAM
Lansman	20	$19	$4	$23/ay
Büyüme	100	$19	$20	$39/ay
Scale	500	$100	$100	$200/ay

Kâra Geçiş

10 paying müşteri ($29/ay) = $290 MRR. Tüm maliyetler karşılanır.

Bölüm 10: Fiyatlandırma

Tier	Fiyat	Env	Step	Instance
Free	$0	3/ay	30K/ay	1
Starter	$29/ay	20/ay	500K/ay	10
Pro	$99/ay	∞	5M/ay	100
Scale	$349/ay	∞	50M/ay	1000

Bölüm 11: Environment Builder

Konsept

Lovable/Bolt.new ama RL environment için. Kullanıcı ne istediğini anlatır, platform anında üretir ve dashboard'da gösterir. Kullanıcı AI ile konuşarak iterate eder: "reward'ı değiştir", "engel sayısını artır", "observation'a şunu ekle". Beğenince GitHub'ına çeker. Opsiyonel olarak: "Bu env'de agent eğit" butonu ile platform agent'ı da eğitir.

Konsept

Lovable/Bolt.new ama RL environment için. Kullanıcı ne istediğini anlatır, platform anında üretir ve dashboard'da gösterir. Kullanıcı AI ile konuşarak iterate eder: "reward'ı değiştir", "engel sayısını artır", "observation'a şunu ekle". Beğenince GitHub'ına çeker. Opsiyonel olarak: "Bu env'de agent eğit" butonu ile platform agent'ı da eğitir.

11.1 Akış

```
ENVIRONMENT BUILDER AKISI:

=== PHASE 1: ENV TASARLA (Lovable modeli) ===

Kullanici: 'Drone navigasyon ortami istiyorum,
            10x10 grid, rastgele engeller'
    |
    v
Architect Agent: Env uretir (30 sn)
    |
    v
Dashboard: Env ozellikleri gosterilir
  - Observation space: Box(shape=[104])
  - Action space: Discrete(4)
  - Reward: distance_to_goal - collision_penalty
  - Episode: max 200 step
  - State Viewer: agent pozisyonu, engeller, hedef
    |
    v
Kullanici AI ile iterate eder (sohbet):
  'Engel sayisini %30'a cikar'  -> Agent gunceller
  'Reward'a yakit tuketimi ekle' -> Agent gunceller
  'Observation'a ruzgar yonu ekle' -> Agent gunceller
  'Difficulty artir'             -> Agent gunceller
  Her degisiklik aninda dashboard'da gorulur.
    |
    v
```

```
Kullanici begendi -> 'GitHub'a cek' butonu
  -> Otomatik private repo olusturulur:
     /env/drone_nav_env.py    (Gymnasium kodu)
     /env/env_config.json     (parametreler)
     /README.md               (dokumantasyon)
     /requirements.txt        (bagimliliklar)
     /examples/test_env.py    (test kodu)

=== PHASE 2: AGENT EGIT (opsiyonel buton) ===

Kullanici: 'Bu env'de agent egit' butonuna basar
    |
    v
Platform: Algoritma secer (PPO/SAC/DQN)
  -> SB3 ile egitir (5dk - 2 saat)
  -> Sonuclari dashboard'da gosterir
    |
    v
Egitilmis model GitHub repo'ya eklenir:
  /model/best_model.zip
  /logs/training_curve.json
  /examples/run_agent.py
```

11.2 Iterate / Konuşma Sistemi

Environment Builder'ın temel farkı: tek seferlik üretim değil, çok turlu sohbet ile iteratif tasarım.

11.2.1 Conversation Agent Prompt

Kullanıcı environment üretildikten sonra sohbet ile değişiklik istediğinde, Architect Agent'a ek prompt eklenir:

```
CONVERSATION MODE PROMPT (system prompt'a eklenir):

Kullanici mevcut bir environment'i degistirmek istiyor.

MEVCUT ENVIRONMENT KODU:
{current_env_code}

MEVCUT ENVIRONMENT SPEC:
{current_env_spec_json}

KULLANICI ISTEGI:
{user_message}

KURALLAR:
```

```
1. Sadece istenen degisikligi yap. Gereksiz degisiklik YAPMA.
2. Mevcut observation/action space'i KORUMA - ekle veya degistir.
3. Degisiklik sonrasi TUM testlerin gecmesi gerekir.
4. Degisikligi acikla: ne degisti, neden, etkisi ne.
5. Breaking change varsa (obs/action space boyutu degisti)
   kullaniciyi UYAR.

CIKTI FORMATI:
1. DEGISIKLIK_OZETI: Tek cumle ne degisti
2. GUNCEL_KOD: Tum environment kodu (tam, calisir)
3. GUNCEL_SPEC: Guncellenmis spec JSON
```

11.2.2 Desteklenen Değişiklik Tipleri

Kullanıcı Komutu	Agent Ne Yapar	Örnek
Reward değiştir	Reward fonksiyonunu günceller	"Yakıt tüketimi ekle" → reward -= fuel_cost
Observation ekle/çıkar	Obs space'i günceller	"Rüzgar yönü ekle" → obs'a wind_dir eklenir
Action değiştir	Action space'i günceller	"Çapraz hareket de ekle" → Discrete(4) → Discrete(8)
Parametre ayarla	Sabit değerleri değiştirir	"Engel %30 olsun" → obstacle_ratio=0.3
Difficulty ayarla	Zorluğu artır/azalt	"Daha zor yap" → daha fazla engel, daha kısa timeout
Fizik değiştir	Transition dynamics günceller	"Yerçekimi ekle" → physics model güncellenir
Termination değiştir	Episode bitiş koşulları	"3 çarpma = ölüm" → collision_count >= 3 → term
Geri al	Son değişikliği geri alır	"Geri al" → önceki versiyona rollback

11.2.3 Dashboard Canlı Görünüm

Her değişiklik sonrası dashboard anında güncellenir:

```
ENVIRONMENT BUILDER DASHBOARD:
+================================================================+
| RLFORGE Environment Builder           [GitHub'a Cek] [Egit]   |
+================================================================+
|                                                                |
| +-- SOHBET (sol panel) ---+  +-- ENV DETAY (sag panel) -----+ |
| | Kullanici: Drone nav    |  | Ad: drone-nav-v3             | |
| |   ortami istiyorum      |  | Obs: Box(104) float32        | |
| |                         |  | Act: Discrete(8)             | |
| | AI: Urettim! 10x10      |  | Reward: goal - fuel - coll   | |
| |   grid, 4 yonlu hareket |  | Episode: max 200 step        | |
| |                         |  | Versiyon: v3 (2 degisiklik)  | |
```

```
| | Kullanici: Capraz       |  +-----------------------------+ |
| |   hareket de ekle       |  |                               |
| |                         |  | +-- STATE VIEWER -------------+|
| | AI: Eklendi! Discrete(4)|  | | Obs degerler tablosu        ||
| |   -> Discrete(8) oldu.  |  | | Reward grafik               ||
| |   Test gecti.           |  | | Son 10 step action log      ||
| |                         |  | +-----------------------------+|
| | Kullanici: Reward'a     |  |                               |
| |   yakit tuketimi ekle   |  | +-- VERSIYON GECMISI --------+|
| |                         |  | | v1: ilk uretim             ||
| | AI: Eklendi! Her step   |  | | v2: engel %30              ||
| |   -0.01 fuel cost.      |  | | v3: capraz hareket eklendi ||
| |   Test gecti.           |  | +-----------------------------+|
| +-------------------------+                                   |
+=================================================================+
```

11.3 GitHub Export

Kullanıcı “GitHub’a çek” butonuna bastığında:

1. **Private repo oluşturulur:** rlforge-envs/{kullanici}/{env-adi}
2. **Tüm versiyonlar commit edilir:** v1, v2, v3... her iterate bir commit
3. **README otomatik üretilir:** Env özellikleri, kurulum, kullanım kodu
4. **ZIP download da mümkün:** GitHub hesabı yoksa

11.4 Agent Eğitim Butonu (Opsiyonel)

Dashboard’da “Bu env’de agent eğit” butonu. Tıklayınca:

1. **Algoritma otomatik seçilir:** Discrete action → DQN/PPO, Continuous → SAC
2. **SB3 ile eğitim başlar:** Progress bar, anlık reward grafik
3. **Sonuçlar dashboard’da:** Mean reward, başarı%, training curve
4. **Model repo’ya eklenir:** /model/best_model.zip + /examples/run_agent.py + /logs/

```
EGITIM SONRASI GITHUB REPO YAPISI:

rlforge-envs/user/drone-nav/
|-- env/
|   |-- drone_nav_env.py      # Gymnasium kodu (iterate edilmis)
|   |-- env_config.json       # Parametreler
```

```
|-- model/                    # (egitim yapildiysa)
|   |-- best_model.zip        # Egitilmis SB3 model
|   |-- checkpoints/          # Ara checkpoint'ler
|-- logs/                     # (egitim yapildiysa)
|   |-- training_curve.json
|   |-- eval_results.json
|-- examples/
|   |-- test_env.py           # Env'i test et
|   |-- run_agent.py          # (egitim yapildiysa) Modeli calistir
|   |-- continue_training.py  # (egitim yapildiysa) Egitimi devam ettir
|-- README.md                 # Otomatik uretilmis dokumantasyon
|-- requirements.txt
```

11.5 Architect Agent Ek Prompt'ları (Environment Builder Modu)

Architect Agent'ın Environment Builder modunda kullandığı ek skill'ler:

Skill: Iterate / Değişiklik Yönetimi

```
ITERATE SKILL PROMPT:

Environment degisiklik yonetimi kurallari:
1. Her degisiklik MINIMAL olmali - sadece istenen seyi degistir
2. Degisiklik sonrasi BUTUN testleri tekrar calistir
3. Breaking change (obs/action boyut degisimi) varsa:
   - Kullaniciyi uyar: 'Action space 4'ten 8'e cikti,
     mevcut agent'lar calismaz'
   - Kullanicidan onay iste
4. Her degisiklik icin version bump yap (v1 -> v2 -> v3)
5. Degisiklik ozeti yaz: ne degisti, neden, etkisi
6. Geri alma destekle: 'geri al' -> onceki versiyona don
7. Degisiklikler ATOMIK olmali - ya tamami uygulanir ya hic
```

Skill: Dashboard Sync

```
DASHBOARD SYNC SKILL PROMPT:

Her degisiklik sonrasi dashboard'a su bilgileri gonder:
1. SPEC_UPDATE: Guncellenmis env spec JSON
2. CODE_UPDATE: Guncellenmis Python kodu
3. CHANGE_SUMMARY: Tek cumle degisiklik ozeti
4. VERSION: Yeni versiyon numarasi
5. TEST_RESULTS: 8 testin sonuclari (pass/fail)
6. BREAKING_CHANGE: bool - obs/action space degisti mi?
7. WARNINGS: Kullaniciya gosterilecek uyarilar
```

11.6 Fiyatlandırma (Environment Builder)

Tier	Env Üretim	Iterate (sohbet)	Agent Eğitim	GitHub Export
Free	3 env/ay	10 iterate/env	1 eğitim/ay	Sınırsız
Starter ($29)	20 env/ay	Sınırsız	5/ay	Sınırsız
Pro ($99)	Sınırsız	Sınırsız	20/ay	Sınırsız
Scale ($349)	Sınırsız	Sınırsız	100/ay	Sınırsız

Bölüm 12: Agent Builder (Lovable for Agents)

> **Konsept**
> Lovable ama RL agent'ı için. Kullanıcı agent'ını AI ile konuşarak tasarlar: ne tür policy, hangi algoritma, nasıl bir strateji. Sonra environment'a bağlar, eğitir, ve sürekli geliştirir. Agent bir kere eğitilip bırakılmaz — yaşayan, büyüyen, adapte olan bir varlık olur.

12.1 Tam Akış

```
AGENT BUILDER AKISI:

=== 1. OLUSTUR (Lovable modeli) ===

Kullanici: 'Trading agent'i istiyorum. Portfoy yonetimi,
            risk-averse, 5 hisseli.'
    |
    v
Platform: Policy network olusturur
  - Algoritma: PPO (otomatik secim veya kullanici tercihi)
  - Network: MLP [256, 256] (domain-based default)
  - Hiperparametreler: domain-optimized defaults
    |
    v
Dashboard: Agent ozellikleri gosterilir
  - Algoritma, network yapisi, hiperparametreler
  - Henuz egitilmemis (random weights)
    |
    v
Kullanici AI ile iterate eder (sohbet):
  'Daha buyuk network olsun'     -> 256,256 -> 512,512
  'SAC kullan PPO yerine'        -> algoritma degisir
  'Learning rate dusur'          -> 3e-4 -> 1e-4
  'Custom reward shaping ekle'   -> reward wrapper eklenir

=== 2. BAGLA ===

Kullanici agent'i bir environment'a baglar:
  a) Environment Builder'da tasarladigi env
  b) Katalogdan hazir env
  c) API ile disaridan env

=== 3. EGIT ===

Ilk egitim calisir:
```

```
  - Progress bar, anlik reward grafik
  - Sonuclar: mean reward, success rate, training curve
  - Model GitHub repo'ya kaydedilir (v1)

=== 4. SUREKLI GELISTIR ===

Agent bir kere egitilip birakilmaz:
  'Daha zor env'de devam et'     -> curriculum learning
  'Farkli env'de dene'           -> transfer testi
  'Reward fonksiyonunu degistir'  -> fine-tune
  'Research Lab ne diyor?'        -> analiz + oneri uygula
  Her egitim = yeni commit. Performans gecmisi gorulur.

=== 5. SAHIPLEN ===

Agent'in GitHub repo'su:
  /agent/config.yaml        # algoritma + hiperparametreler
  /agent/policy.py          # custom policy (varsa)
  /models/v1/model.zip      # ilk egitim
  /models/v2/model.zip      # curriculum sonrasi
  /models/v3/model.zip      # fine-tune sonrasi
  /logs/                    # tum egitim gecmisi
  /envs/                    # bagli environment'lar
  README.md                 # otomatik dokumantasyon
```

12.2 Agent Oluşturma (AI ile Iterate)

Kullanıcı agent'ını AI ile konuşarak tasarlar:

12.2.1 Agent Builder Conversation Prompt

```
AGENT BUILDER CONVERSATION PROMPT:

Sen Kuaila.ai platformunun Agent Builder'isin.
Kullanici bir RL agent'i olusturmak veya degistirmek istiyor.

MEVCUT AGENT CONFIG (varsa):
{current_agent_config_yaml}

KULLANICI ISTEGI:
{user_message}

KURALLAR:
1. Stable-Baselines3 uyumlu agent config uret
2. Desteklenen algoritmalar: PPO, SAC, DQN, A2C, TD3, DDPG
3. Algoritma secimi icin rehber:
```

```
   - Discrete action -> DQN veya PPO
   - Continuous action -> SAC veya TD3
   - Genel amacli / bilmiyorsan -> PPO
4. Network boyutu domain'e gore:
   - Basit (grid, cartpole): [64, 64]
   - Orta (trading, navigasyon): [256, 256]
   - Karmasik (multi-agent, robotik): [512, 512]
5. Her degisikligi acikla: ne degisti, neden, beklenen etki
6. Breaking change varsa uyar (algoritma degisimi = sifirdan egitim)

CIKTI: YAML formatinda agent config + degisiklik ozeti
```

12.2.2 Agent Config Yapısı

```yaml
# agent_config.yaml
name: trading-agent-v2
algorithm: PPO
policy: MlpPolicy
network:
  pi: [256, 256]        # policy network
  vf: [256, 256]        # value function network
hyperparameters:
  learning_rate: 0.0003
  batch_size: 256
  n_steps: 2048
  gamma: 0.995
  ent_coef: 0.01
  clip_range: 0.2
training:
  total_timesteps: 2000000
  eval_freq: 50000
  n_eval_episodes: 20
  checkpoint_freq: 100000
connected_envs:
  - stock-trading-5-v3
  - stock-trading-crypto-v1
```

12.2.3 Desteklenen Değişiklikler

Kullanıcı Komutu	Agent Builder Ne Yapar	Örnek
Algoritma değiştir	Tamamen farklı algoritma	“SAC kullan” → PPO → SAC (⚠️ sıfırdan eğitim)
Network büyüt/küçült	Policy/value network katmanları	“Daha büyük” → [256,256] → [512,512]
Hiperparametre ayarla	LR, batch, gamma vb.	“LR düşür” → 3e-4 → 1e-4
Custom policy ekle	Kullanıcının kendi network yapısı	“LSTM katmanı ekle” → RecurrentPPO

Kullanıcı Komutu	Agent Builder Ne Yapar	Örnek
Reward wrapper ekle	Agent tarafında reward shaping	"Reward'ı normalize et" → RunningMeanStd
Observation wrapper	Agent tarafında obs işleme	"Frame stack ekle" → FrameStack(4)
Geri al	Son değişikliği geri al	"Geri al" → önceki config

12.3 Sürekli Geliştirme Sistemi

Agent Builder'ın en güçlü özelliği: agent bir kere eğitilip bırakılmaz. Yaşayan, büyüyen, adapte olan bir varlık olur.

12.3.1 Sürekli Geliştirme Modları

Mod	Ne Yapar	Ne Zaman Kullanılır	Teknik Detay
Continue Training	Aynı env'de eğitime devam et	Agent henüz yeterince iyi değil	model.learn(additional_steps) — mevcut ağırlıklar korunur
Curriculum Learning	Giderek zorlalan env'lerde eğit	Agent kolay görevde ustalaştı	Architect Agent env difficulty'yi otomatik artırır
Fine-Tune	Değişen koşullara adapte et	Env değişti, reward değişti	Mevcut model + düşük LR + kısa eğitim
Transfer Test	Farklı env'de performansı test et	Generalization ölçmek	Mevcut modeli yeni env'de eval (eğitim yok)
Transfer Train	Farklı env'de eğitime devam et	Yeni domain'e adapte etmek	Mevcut ağırlıklar + yeni env + eğitim
Research-Guided	Research Lab önerilerini uygula	Analiz agentı sorun tespit etti	Lab'ın önerdiği değişiklik otomatik uygulanır

12.3.2 Agent Yaşam Döngüsü

```
AGENT YASAM DONGUSU:

Olustur (AI ile tasarla)
  |
  v
Ilk Egitim (env-v1, 1M step)
  |  Sonuc: mean_reward=23, success=45%
  v
Curriculum (env-v1-hard, 500K step)
  |  Sonuc: mean_reward=31, success=62%
  v
Fine-Tune (reward degisti, 200K step)
  |  Sonuc: mean_reward=38, success=71%
```

```
  v
Transfer Test (farkli env'de eval)
  |  Sonuc: mean_reward=15 (dusuk -> transfer train gerekli)
  v
Transfer Train (yeni env, 500K step)
  |  Sonuc: mean_reward=29, success=58%
  v
Research Lab Onerisi: 'Reward hacking var, step 200K'de'
  |  -> Anti-exploit reward eklendi, fine-tune
  v
... devam eder. Agent surekli gelisir.

Her adim = GitHub repo'da yeni commit.
Performans gecmisi = tum versiyonlarin karsilastirmasi.
```

12.4 Agent Dashboard

Agent Builder'ın dashboard'ı agent'ın tüm yaşamını gösterir:

- **Sol panel:** AI sohbet — agent'ı tasarla, değiştir, geliştirme komutları ver
- **Sağ panel — Agent Spec:** Algoritma, network yapısı, hiperparametreler, bağlı env'ler
- **Sağ panel — Performans Geçmişi:** Tüm eğitimlerin reward curve'leri üst üste (v1, v2, v3...)
- **Sağ panel — Versiyon Karşılaştırma:** v1 vs v3: mean reward, success rate, episode length yan yana
- **Sağ panel — Transfer Haritası:** Agent hangi env'lerde eğitildi, hangisinde nasıl performans gösterdi
- **Butonlar:** "Eğitime devam", "Farklı env'de dene", "Research Lab'a sor", "GitHub'a çek"

12.5 Research Lab Entegrasyonu

Agent Builder, Research Lab ile doğrudan entegre çalışır:

1. **Analysis Agent:** "Neden %60'ta takılıyorum?" → "Reward hacking var, step 200K'de agent exploit buluyor. Anti-exploit reward eklenmeli."
2. **Experiment Agent:** "PPO vs SAC hangisi daha iyi bu env'de?" → İkisiyle de eğit, karşılaştır, öner.
3. **Benchmark Agent:** "Agent'ım sektör ortalamasına göre nasıl?" → Leaderboard pozisyonu ve karşılaştırma.

4. **Literature Agent:** "Bu domain'de en iyi ne çalışıyor?" → Paper'lardan algoritma/hiperparametre önerileri.

12.6 Fiyatlandırma (Agent Builder)

Tier	Agent Oluşturma	Iterate	Eğitim	Sürekli Geliştirme
Free	1 agent	10 iterate	1 eğitim/ay	Yok
Starter ($29)	5 agent	Sınırsız	5/ay	Continue + Fine-tune
Pro ($99)	Sınırsız	Sınırsız	20/ay	Tüm modlar
Scale ($349)	Sınırsız	Sınırsız	100/ay	Tüm + öncelikli compute

Yapışkanlık

Agent Builder platformun en güçlü retention mekanizmasıdır. Kullanıcının agent'ı büyüdükçe (v1 → v2 → v10), performans geçmişi biriktikçe, farklı env'lere transfer edildikçe switching cost katlanarak artar. Agent = kullanıcının dijital varlığı.

Bölüm 13: AI Research Lab

> **Konsept**
> Platformun içinde beş uzman AI agent'tan oluşan araştırma ekibi. Literür tarama, paper replikasyonu, deney yönetimi, sonuç analizi ve benchmark.

13.1 Konsept

RLForge sadece araç değil, araştırma ortağı olur. Platformun içinde AI agent'lardan oluşan bir araştırma ekibi bulunur.

13.2 Agent Ekibi

Agent	Görev	Girdi	Çıktı
Literature Agent	Paper tarama, özetleme, karşılaştırma	Konu/domain	Paper özet tablosu, env+algoritma+metrik listesi
Replication Agent	Paper'daki setup'ı replike et	Paper PDF	Environment + eğitim + baseline karşılaştırma
Experiment Agent	Hipotez test et, A/B deney	Env + değişiklik talimatı	Karşılaştırma raporu (reward curve overlay)
Analysis Agent	Eğitim sonuçlarını analiz et	Training logları	Neden takıldı, ne değiştirelim, öneri
Benchmark Agent	Farklı algoritma/env kombinasyonlarını karşılaştır	Env seti + algoritma seti	Benchmark tablosu + sıralama

13.3 Kullanıcı Senaryosu

```
KULLANICI: 'Multi-agent cooperation konusunda ne var?'

LITERATURE AGENT:
  -> Son 20 paper'i tarar
  -> Kullanilan env'leri, algoritmalari, metrikleri cikarir
  -> Ozet tablo sunar

KULLANICI: 'Su paper'daki setup'i replike et' (PDF yukler)

REPLICATION AGENT:
  -> Paper'i okur, env spec cikarir
  -> Architect Agent ile env uretir
```

```
  -> Paper'daki algoritma ile egitir
  -> Sonuclari paper'daki baseline ile karsilastirir

KULLANICI: 'Reward fonksiyonunu degistirip dene'

EXPERIMENT AGENT:
  -> Orijinal ve degistirilmis reward ile iki egitim calistirir
  -> Reward curve'leri ust uste koyar
  -> Istatistiksel karsilastirma sunar

ANALYSIS AGENT:
  -> 'Reward hacking var gibi gorunuyor, step 200K'de agent
     exploit buluyor. Reward fonksiyonuna anti-exploit eklenmeli.'
```

Bölüm 13: Yol Haritası

Faz 1 (Hafta 1-3): Mod A Foundation

- Architect Agent v1 (tüm prompt/skill/tool bu dokümandaki gibi)
- REST API + Python SDK
- 5 hazır env + custom env generation
- Tek sunucu + State Viewer

Faz 2 (Hafta 4-6): Mod A Olgunlaşma

- Vectorized API + ödeme (Stripe) + dashboard
- Architect Agent v2: otonom iyileştirme

Faz 3 (Hafta 7-9): Environment Builder

- Builder UI: sohbet + env detay + state viewer + iterate + GitHub export + Canvas viz

Faz 4 (Hafta 10-12): Agent Builder + Training

- Agent Builder: AI ile agent tasarla + iterate + env'e bağla + eğit
- Sürekli geliştirme: continue, curriculum, fine-tune, transfer
- Agent Training butonu (tüm ürünlere entegre)

Faz 5 (Ay 4-6): Research Lab + Growth

- AI Research Lab: 5 agent (Literature, Replication, Experiment, Analysis, Benchmark)
- Paper upload + 20+ env şablonu + Leaderboard + WebSocket

Bölüm 14: Riskler

Risk	Etki	Mitigasyon
LLM env kalitesi	Yüksek	Iteratif düzeltme + 8 test
Reward hacking	Yüksek	Reward sanity + anomali tespiti
Compute maliyeti	Orta	Idle timeout, LRU, limit
Güvenlik	Kritik	No egress, sandbox, code scan
Büyük oyuncu	Yüksek	Hız + niş + data moat

EK A: Landing Page Metinleri

A.1 — Hazır RL Environment API

Başlık: Production-Ready RL Environments. Tek API Call Uzaklığında.

Alt başlık: Haftalar süren environment geliştirmeyi unutun. RLForge'un hazır, test edilmiş, Gymnasium-uyumlu ortamlarını API üzerinden anında kullanmaya başlayın.

Nedir?

RLForge, farklı domain'lerde (finans, oyun, lojistik, robotik, optimizasyon) hazır RL environment'ları sunan bir platformdur. Her environment, Gymnasium v0.29+ standardında üretilmiş, 8 farklı testten geçmiş ve Docker container içinde izole olarak çalışır.

Nasıl Çalışır?

1. **Kayıt ol,** API key al. 30 saniye.
2. **Environment seç,** katalogdan domain ve zorluk seviyesine göre filtrele.
3. **SDK ile bağlan:** rlforge.make('trading-5stock') — tek satır.
4. **Agent'ını eğit:** env.step(action) — Stable-Baselines3, RLlib, CleanRL, herhangi bir framework.

Ne Sunar?

- **Sıfır kurulum:** Docker, Kubernetes, GPU — hiçbiriyle uğraşma. pip install rlforge, başla.
- **Gymnasium uyumlu:** Mevcut RL kodun değişmeden çalışır.
- **Test edilmiş:** Her env 8 otomatik testten geçmiş: syntax, API uyumu, reward sanity, determinism, performance, memory.
- **Sürekli iyileşen:** Architect Agent her ortamı sürekli izler, reward hacking yakalar, bug'ları fix'ler, difficulty'yi ayarlar.
- **Free tier:** 3 environment, 30K step/ay. Kredi kartı gerekmez.

```
# 3 satirda baslat
import rlforge
env = rlforge.make('gridworld-maze-v1', api_key='YOUR_KEY')
```

```
obs, info = env.reset()
```

A.2 — Custom RL Environment Generation

Başlık: Hayalindeki RL Ortamını Anlat. 30 Saniyede Hazır.

Alt başlık: Kod yazmadan, Gymnasium bilmeden, Docker kurmadan — sadece ne istediğini yaz. RLForge'un AI Architect Agent'ı environment'ını saniyeler içinde üretir, test eder ve API olarak yayınlar.

Nedir?

RLForge'un Architect Agent'ı, doğal dil açıklamanızı çalışır bir RL environment'a dönüştüren bir AI'dır. "5 hisseli borsa trading ortamı, komisyon %0.1" yazın — 30 saniye sonra test edilmiş, deploy edilmiş, API'si açık bir Gymnasium environment'ınız olsun.

Nasıl Çalışır?

1. **Anlat:** "Drone engelden kaçsın, 10x10 grid, rastgele engeller" — sadece yaz.
2. **Architect Agent üretir:** AI, doğal dilinizi analiz eder. Domain'a uygun observation space, action space, reward function ve fizik kurallarını belirler. Gymnasium-uyumlu Python kodu yazar.
3. **Otomatik test:** 8 farklı test (syntax, API uyumu, reward sanity, determinism, performance, memory) otomatik çalışır. Başarısızsa agent kodu düzeltir ve tekrar test eder.
4. **Anında yayınla:** Test geçen environment Docker container'a paketlenir ve API endpoint'i anında açılır. rlforge.make('senin-env') ile kullanmaya başla.

Ne Sunar?

- **Kod yazmadan environment:** Gymnasium syntaxı, reward engineering, observation normalization — hiçbirini bilmene gerek yok.
- **5 domain desteği:** Finans, oyun, robotik, lojistik, optimizasyon — her domain için uzmanlaşmış AI skill'leri.
- **Kendini iyileştiren ortamlar:** Architect Agent, ortamı yaratmakla kalmaz — sürekli izler. Reward hacking yakalar, bug fix'ler, difficulty ayarlar.

- **Fork & özelleştir:** Mevcut bir ortamı al, üzerine değişiklik yap. “Trading ortamını al ama kripto için uyarla.”
- **NVIDIA Eureka tabanlı:** Akademik olarak kanıtlanmış LLM-based environment generation. %83 oranında insan uzmanları geçen kalite.

```
# Custom env olustur ve kullan
import rlforge

# 1. Env'ini tanimla
env_id = rlforge.generate(
  description='Drone engelden kacsin, 10x10 grid, rastgele engeller',
  domain='robotics'
)

# 2. Kullanmaya basla
env = rlforge.make(env_id, api_key='YOUR_KEY')
obs, info = env.reset()
```

A.3 — Auto Research Lab

Başlık: RL Araştırmanız İçin AI Ekibiniz. Paper’dan Deneyime, Dakikalar İçinde.

Alt başlık: Paper okumak, repo bulmak, dataset indirmek, environment kurmak için saatler harcamayı bırakın. RLForge’un AI araştırma ekibi, paper’ınızı yükleyin, deneylerinizi çalıştırsın, sonuçları karşılaştırsın.

Nedir?

RLForge Research Lab, platformun içinde çalışan bir AI araştırma ekibidir. Beş uzman agent’tan oluşur: literür tarama, paper replikasyonu, deney yönetimi, sonuç analizi ve benchmark. Her biri otonom çalışır, birbirleriyle koordine olur.

Nasıl Çalışır?

1. **Paper yükle:** PDF’i sürükle-bırak. Literature Agent paper’ı okur: environment, algoritma, metrikler, baseline sonuçları çıkarır.

2. **Replike et:** Replication Agent, paper'daki setup'ı otomatik oluşturur. Architect Agent environment'ı üretir, SB3 ile eğitir, sonuçları paper'ın baseline'ıyla karşılaştırır.
3. **Dene:** "Reward fonksiyonunu değiştir" de. Experiment Agent orijinal ve değiştirilmiş versiyonla iki eğitim çalıştırır, sonuçları üst üste koyar.
4. **Analiz al:** Analysis Agent: "Step 200K'de reward hacking var, agent exploit buluyor. Anti-exploit eklenmeli." — somut, uygulanabilir öneriler.

Ne Sunar?

- **Paper'dan deneyime 15 dakikada:** PDF yükle, bekle, sonuçları gör. Repo bulmak, dataset indirmek, environment kurmak yok.
- **Otomatik literür tarama:** "Multi-agent cooperation'da son 20 paper ne kullanmış?" — env, algoritma, metrik tablosu anında.
- **A/B deneyleri tek tıkla:** "Reward'ı değiştir ve karşılaştır" — iki eğitim paralel çalışır, grafik üst üste gelir.
- **Akıllı analiz:** Neden takıldın, ne değiştirmelisin, hangi hiperparametre önemli — AI tabanlı öneriler.
- **Tüm sonuçlar GitHub'da:** Her deney otomatik repo olur. Tekrarlanabilir, paylaşılabilir, yayınlanabilir.

MVP KAPSAMI — SADECE BUNLARI KODLA:

- Architect Agent v1 (system prompt + 5 hazır skill + dinamik skill generation + 8 test pipeline + hata düzeltme döngüsü)
- REST API: /v1/envs, /v1/sessions/create, reset, step, close
- Python SDK: pip install rlforge → rlforge.make('env-id')
- 5-10 hazır environment (gridworld, cartpole, maze, taxi, mountain car)
- Custom env generation (NL → spec → kod → test → deploy)
- State Viewer dashboard
- Environment Builder (Lovable modeli: AI ile iterate, dashboard, GitHub export)
- Agent Training ("Bu env'de agent eğit" butonu, SB3)

- Paper Upload (PDF → env spec)
- AI Research Lab (5 agent: Literature, Replication, Experiment, Analysis, Benchmark)
- Canvas 2D render
- WebSocket modu
- Leaderboard ve benchmark
- Mod B: Agent Upload
- Hosting: Hetzner + Docker + Nginx + FastAPI + PostgreSQL + Redis + Cloudflare
- Güvenlik: container izolasyonu, no egress, resource limits, kod tarama

KAPSAM DIŞI — KODLANMAYACAK

Kubernetes: gereksiz, tek sunucu + Docker yeterli. gRPC: gereksiz, HTTP REST + WebSocket yeterli. Bu ikisi kapsam dışıdır, kodlanmayacaktır.